



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE:CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Code of Conduct:

I R Nithya Priya / 192421119 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question and Answers

1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer:

Testing is a planned and measurable discipline, not random trial and error. The redesigned approach follows a defined cycle: plan, analyse requirements, design test cases, execute, report defects, and measure results.

Plan: define scope, risks, schedule and exit criteria in a test plan. Design: derive test cases from requirements using equivalence partitioning, boundary value analysis and decision tables. Execute and report: run cases in a controlled environment and log defects with a standard template. Measure: track coverage, defect density and pass rate; improve each release.

Every requirement is traced to at least one test case, so coverage gaps are visible. Techniques target error-prone areas such as boundaries, so more defects are found with fewer tests. Tests are repeatable and can be automated for regression. Results are comparable across releases, which gives consistent quality.

Example: for a sign-up form that accepts ages 18 to 60, structured design gives the tests 17, 18, 60 and 61. Random testing with ages like 25 or 30 would miss an error such as age > 18 instead of age >= 18.

2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer:

A defined process has fixed stages: test planning and control, analysis and design, implementation, execution, exit-criteria evaluation and closure. Each stage has owners, templates and entry and exit criteria.

Consistency: every release is tested in the same planned way, so nothing is skipped. Coverage: requirements such as limits, fees and rollbacks are traced to test cases. Early detection: reviews and unit tests find defects before they reach production. Control of risk: high-risk functions like fund transfer are tested first and deeper. Auditability: documented results satisfy regulatory and audit needs.

A transfer fails after the debit but before the credit. A defined process includes failure-recovery tests, so the rollback defect is found before release. A daily limit of Rs. 1,00,000 is tested with 99,999, 1,00,000 and 1,00,001, finding boundary errors systematically. Regression tests re-run after each fix so that new changes do not break earlier working features.

Benefit summary: fewer production failures, lower rework cost and higher customer trust.

3. During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer:

Requirements: review requirements for clarity and testability; define acceptance criteria. Design: review design and interfaces; prepare the test plan, strategy and test data. Coding: unit testing, code review and static analysis. Integration: test interfaces between catalogue, cart, payment and shipping modules. System: end-to-end functional, performance, security and compatibility tests. Acceptance and maintenance: user acceptance testing, then regression testing for every change.

Every phase has defined testing activities and exit criteria, so no phase is skipped. Defects are caught in the phase where they are introduced, when they are cheapest to fix. Integration and system defects reduce because interfaces are planned and tested, not assumed. Standard documents and metrics make progress and risk visible to management.

Example: a review of the cart-payment interface finds that the order total is passed as text in one module and as a number in another. Integration testing catches this mismatch before system testing, avoiding wrong totals at checkout.

4. A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer:

Error: a human mistake, such as a developer misunderstanding a requirement. Defect (bug or fault): a flaw in the code or document caused by an error. Failure: the visible deviation from expected behaviour when the defect is executed.

Expected: a valid user logs in with the correct password. Actual: the system says "Invalid password". This mismatch observed during testing is a failure. The cause in the code, for example comparing the

password without trimming spaces, is the defect. The developer's wrong assumption that no spaces will ever occur is the error.

Error leads to defect, and defect leads to failure when the faulty code is executed. A defect may stay hidden if the conditions that trigger it never occur.

Example: the requirement says "lock the account after 3 wrong attempts". The developer writes attempts > 3 (error). The code allows four attempts (defect). The tester enters four wrong passwords and the account is not locked after three (failure). With only two wrong attempts the failure would never appear.

5. A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer:

The early testing principle says that testing activities should start as early as possible in the SDLC, so defects are found when they are cheapest to fix.

Testers review requirements and design documents for ambiguity and missing cases. Test cases and test data are prepared in parallel with development. Developers perform unit tests and code reviews before integration. Static testing (reviews, walkthroughs, inspections) is used before any code runs.

The cost of fixing a defect rises sharply in later phases, so early detection reduces cost and rework. Requirements become clear and testable, so fewer defects are injected. Test execution time is shorter because test design is already complete.

Example: during requirement review, a tester asks whether the rule "discount for orders above Rs. 500" includes exactly Rs. 500. The ambiguity is settled before coding. Found after release, the same issue would need code, test and documentation changes.

6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer:

Review requirements for completeness, consistency, ambiguity and testability. Ask clarifying questions, e.g., "What is the valid range of a dosage value?" or "Who can edit a patient record?". Define acceptance criteria and identify testable conditions for each requirement. Build a requirement

traceability matrix so each requirement maps to future test cases. Identify risks such as patient safety, data privacy and data integrity.

Defects in requirements are found when the cost of correction is lowest. Ambiguity is removed before design and coding, so fewer defects are injected. Test planning starts early, shortening the later test phase. Better communication between business, developers and testers reduces rework.

Example: the requirement says "alert the doctor for abnormal lab results". The tester asks what counts as abnormal and how the alert is delivered. The clarified rule gives precise test cases and avoids a missed-alert defect that could affect patient safety.

7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer:

Requirements: ambiguous or missing rules (tax slabs, overtime, leave deduction) lead to wrong understanding. Design: the wrong algorithm or data structure is chosen from an unclear requirement. Coding: the developer interprets an unclear rule in a different way, e.g., > instead of >=. Testing: testers derive expected results from the same unclear requirement, so the defect is not detected. Maintenance: changes to rules are not documented, producing new defects.

Studies show that a large share of defects originate in the requirement phase, so unclear requirements are the root cause here.

Written, unambiguous rules with examples, e.g., "Overtime is paid at 1.5 times the hourly rate beyond 8 hours". Requirement reviews with HR and finance and tester participation. Decision tables for rule combinations and acceptance criteria for each rule. Traceability and change control so all changes are reviewed and tested.

Example: a clear rule gives the tester an exact expected result (10 hours overtime at Rs. 200 per hour = Rs. 3,000), so the code can be verified precisely.

8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer:

By type: boundary or input-validation defect (functional/logic class). By severity: critical or major, since the system crashes. By origin: coding phase (off-by-one or missing validation), possibly aided by missing requirements for limits. Justification: the failure occurs only at the edge values of the valid range, which is typical of incorrect relational operators (< versus <=) or unhandled limits.

BVA selects test values at and around the limits of each input range: minimum, just below, just above, maximum and so on. It targets exactly the places where such defects hide.

Example: a quantity field accepts 1 to 100. BVA tests 0, 1, 2, 99, 100 and 101. If the program crashes for 100 because an array of size 100 is indexed at 100, this test would reveal it before release. Normal values such as 10 or 50 would never find it.

9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer:

Find defect-prone modules and design more tests for them (defect clustering). Find frequent defect types, e.g., boundary or validation errors, and add matching techniques such as BVA. Convert each past defect into a regression test case. Use severity and impact data to prioritise test cases. Use escaped defects to find missing scenarios in the earlier test design.

Example: if the repository shows many "missing input validation" defects in forms, the team adds equivalence and boundary tests for every input field in the next release.

Provides data for root cause analysis and preventive action. Reduces repeated defects through regression suites and checklists. Supports metrics such as defect density, defect removal efficiency and reopen rate. Retains organisational knowledge for new team members and future projects.

10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer:

A functional defect occurs when the system does not behave as the requirement specifies. Here the requirement is: dispense cash and reduce the balance by the withdrawn amount only if the balance and limit allow it. If the logic debits the account twice, dispenses cash without checking the balance, or does not roll back when dispensing fails, the function itself is wrong.

Example: a withdrawal of Rs. 2,000 from a balance of Rs. 10,000 leaves a balance of Rs. 6,000, because the debit is applied twice. The defect is in the transaction logic (coding), the failure is the wrong balance, and the severity is critical.

Review requirements and transaction rules and write clear acceptance criteria. Use decision tables and state-transition testing for withdrawal conditions (valid, insufficient balance, over limit, ATM out of cash). Add negative and failure-recovery tests, e.g., power or network failure after debit. Perform unit and integration tests for the debit-dispense-update sequence and code reviews of the transaction logic. Run regression tests after every change.

11. A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer:

Defect clustering states that a small number of modules usually contain most of the defects (the Pareto 80/20 rule).

Analyse defect data from earlier releases and cycles to calculate defect density per module. Rank modules by defect count, complexity, change frequency and business risk. Give top-ranked modules the most test cases, experienced testers, extra reviews and daily regression. Give low-risk modules basic coverage. Re-check regularly because clusters can move.

Example: Payment (60% of defects) is P1, Login is P2, and Profile is P3.

More defects are found per hour of testing effort. Critical modules stabilise earlier, which reduces release delays. Effort is not wasted on stable, low-risk modules.

12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer:

Type: performance defect (non-functional), specifically scalability or load-related. Severity: major to critical, because users are affected at peak time. Priority: high. Likely origin: design or coding phase, e.g., inefficient database queries, missing indexes, no caching, small connection pool, or memory leaks. Detected by: load and stress testing.

The defect is routed to the right team (performance engineers, DBAs) instead of being treated as a functional bug. Severity and priority decide the order of fixing. The type points to the right tests (load, stress, endurance) and tools (JMeter, APM). Root cause data helps prevent similar issues in future releases.

Example: classifying slow checkout under 2,000 users as a database-query performance defect leads to query indexing and caching, and the retest shows a response time of 2 seconds instead of 12.

13. A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer:

Convert every previously fixed defect into a regression test case. Select tests for high-defect modules and frequently changing areas first (risk-based selection). Use defect types to add suitable tests, e.g., boundary tests for earlier boundary defects. Use severity data to prioritise critical-defect tests in smoke and sanity suites. Identify tests that never find defects and review or remove them.

Coverage improves because areas where defects actually occurred are always re-verified. Efficiency improves because the regression suite is focused and not run blindly in full. Automation of defect-based tests reduces manual effort and speeds up each release. Recurring defects are detected immediately when they return.

Example: a past defect where a coupon was applied twice is added as a regression test. In a later release, it fails again after a code change and is caught the same day.

14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer:

Review requirements and interface specifications and clarify doubts with developers. Share test plans, test cases and test data with the team. Report defects with clear steps, expected and actual results and evidence. Take part in design reviews, daily stand-ups and defect triage meetings. Verify fixes and run regression, and give timely feedback.

Interface mismatches (data formats, field lengths, error codes) are found at design time. A common understanding of requirements reduces misinterpretation. Developers know the test conditions in advance and write code that meets them. Fewer integration defects arise, so the integration phase is shorter.

Example: a tester notices in a design review that one module sends the date as DD-MM-YYYY while another expects YYYY-MM-DD. The correction is made before coding, so the integration defect never occurs.

15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer:

Root cause analysis: analyse the defect repository using Pareto and fishbone diagrams to find why defects recur. Reviews and inspections: requirement, design and code reviews with checklists. Standards and training: coding standards, secure coding guidelines and training on common error types. Process improvement: add tests for every fixed defect in an automated regression suite. Static analysis and unit testing: tools catch common errors before testing. Lessons learned: review each release and update the process.

Fewer defects are injected, so less effort is needed to find and fix them. Cost and time of rework fall and releases become more predictable. Quality becomes part of the process and not a final inspection. Customer satisfaction and trust increase.

Example: if input validation defects recur, adding a validation checklist to code review and a standard validation library prevents them at the source.

16. A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

Answer:

Early testing: review requirements and write unit tests early. Defect clustering: focus on crash-prone screens and flows. Exhaustive testing is impossible: use a device and OS matrix based on usage analytics. Pesticide paradox: review and update tests each release. Context dependence: include mobile-specific tests: interruptions (calls, notifications), low battery, poor network, orientation and permissions. Standardise: common templates, a single test management tool and automated regression with Appium.

Every release is tested the same way, so crashes are found before users see them. Crash logs (Crashlytics) feed back into the test suite. Measured coverage and metrics show weak areas. Automated regression prevents old crashes from returning.

Example: a crash occurs when the network drops during login. A standard network-interruption test is added to every release, so the unhandled-exception defect is caught and fixed.

17. A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Answer:

Testing: a planned activity to find failures and defects by running the software against expected results; done mainly by testers; output is a defect report. Debugging: an activity to locate the cause of a reported failure in the code and fix it; done by developers; output is a code fix and root cause.

In an online ordering system, a tester runs a test with a coupon code and observes that the discount is not applied (testing: the failure is detected and reported with steps). The developer reproduces it, uses breakpoints and logs, and finds that the coupon check compares the code case-sensitively (debugging: the defect is located and fixed). The tester then retests and runs regression to confirm the fix and check other coupons.

Testing gives information about quality and risk and shows that defects exist. Debugging removes the cause of defects and improves the code. Together they form the loop test, debug, retest, regression. Confusing them leads to missed related defects and unverified fixes.

18. An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

Answer:

Scope (exhaustive testing is impossible): use analytics to choose top browsers, versions, operating systems and devices (Chrome, Edge, Firefox, Safari; Windows, macOS, Android, iOS). Prioritise (defect clustering and risk): test critical flows first: search, cart, checkout, payment. Design: build a compatibility matrix of browser x OS x device against test cases for layout, functionality and responsiveness. Execute: automated tests with Selenium, Playwright or Cypress, plus manual exploratory checks; cloud tools like BrowserStack for real devices. Report and retest: log browser, version and screenshots; run regression after fixes.

Cause: different rendering engines and unsupported CSS or JavaScript features. Use standard-compliant code, feature detection and progressive enhancement. Use visual regression tools (Percy, Applitools) and run suites in CI on every build. Update the matrix when browsers release new versions.

Example: the checkout button overlaps text in Safari. The matrix test finds it before release, and a CSS fix is applied.

19. A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Answer:

Missing checks on mandatory fields, formats and ranges (pincode, weight, dates). Boundary values not tested (zero or maximum weight). Invalid and special characters or empty inputs not handled. Cross-field rules not validated (delivery date before pickup date). Validation present on the client side only and not on the server side. Requirements did not define validation rules, and tests used only normal data.

Equivalence partitioning and BVA test valid and invalid classes and limits. Negative testing and error guessing verify error messages and safe handling of wrong input. Early reviews of validation rules and unit tests catch defects before integration. Regression tests keep validation working after changes.

Example: a shipment weight field accepts 0.1 to 500 kg. Tests with 0, 0.1, 500, 501, -5 and "abc" reveal that 0 and -5 are wrongly accepted, which would have produced incorrect freight charges.

20. A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer:

Treat testing as an engineering activity: analyse requirements, select test design techniques, write test cases (ID, steps, data, expected result), review them, map them to requirements, execute, log defects and measure coverage.

Ensures all requirements, including negative and boundary cases, are covered. Makes testing repeatable and suitable for automation. Finds more defects with fewer tests by using techniques such as BVA, equivalence partitioning and decision tables. Allows prioritisation by risk when time is limited. Gives evidence of testing for audits and release decisions.

Example: for a password length of 8 to 12 characters, designed tests with lengths 7, 8, 12 and 13 reveal an error such as $\text{length} > 8$, which random tests such as "abc123" would not.

21. A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer:

Early testing: interface and data specifications were not reviewed, so no single source of truth was defined for each data item. Defect clustering: inconsistencies concentrate at interfaces such as Registration to Billing. Context dependence: healthcare needs very high accuracy. Typical causes: different data types, formats or field lengths between modules; wrong interface mapping; duplicate data not synchronised; missing transaction control; concurrent updates.

Integration testing checks data passing between modules. Database testing verifies constraints, referential integrity and ACID properties. End-to-end system tests check that one update appears in all modules. Concurrency and regression tests prevent conflicts and new inconsistencies.

Example: a patient's weight is changed in Nursing. The test confirms that Pharmacy dosage calculation, Billing and the patient summary all show the new value.

22. A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer:

Requirements: review for ambiguity and testability; define acceptance criteria. Design: review design and interfaces; prepare test plan, strategy and data. Coding: write test cases; support unit and integration testing. Testing: execute tests, report defects, confirm fixes and run regression. Deployment: smoke tests and UAT support. Maintenance: regression tests and analysis of production defects.

Defects in requirements and design are found when they are cheapest to fix. Test preparation runs in parallel with development, so there is less delay at the end. Better shared understanding of requirements reduces rework. Collaboration (stand-ups, joint reviews, shift-left practices) improves communication.

Example: a tester reviewing the requirement "free delivery above Rs. 500" asks whether it is before or after discount. The clarification prevents a pricing defect that would otherwise delay release.

23. A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

Answer:

Performance requirements (users, response time) were not defined early. Only functional tests with a few users were run; no load, stress, spike or endurance tests. The test environment and data were not production-like. Risk analysis did not rank peak load as a high-risk scenario. Requirement and risk coverage were not measured.

Coverage of load profiles and critical flows (login, submission, payment) exposes bottlenecks. Load and stress tests find the breaking point and capacity limits before release. Endurance tests find memory leaks; spike tests check sudden surges. Coverage metrics show which scenarios remain untested.

Example: a JMeter test with 3,000 users shows that the database connection pool is exhausted and response time reaches 20 seconds. After increasing the pool and adding indexes, the retest shows a response time of 2 seconds, so the peak-time failure is avoided.

24. A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer:

Use one central tool (Jira, Bugzilla) as the defect repository. Define a standard record: ID, title, module, environment, build, steps, expected and actual result, severity, priority, status, assignee, root cause and attachments. Define a defect life cycle: New, Assigned, Open, Fixed, Retest, Closed, Reopened. Check for duplicates before logging.

Every defect has an owner and status, so none is lost. Developers can reproduce and fix faster with complete information. Analysis is possible: defect density, Pareto charts, trend and aging analysis, and reopen rates. Management can judge release readiness from open defects by severity. Knowledge is kept for future releases and audits.

25. A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer:

Early testing: review transaction rules (limits, fees, rollback) before coding. Risk-based and defect clustering: focus on fund transfer and payment modules. Exhaustive testing impossible: use BVA, equivalence partitioning and decision tables. Pesticide paradox: update tests every release and add each escaped defect. Context dependence: verify ACID properties, security and audit trails. Automation: regression tests for core transactions in CI.

Valid, invalid and boundary transactions are covered systematically. Failure-recovery and concurrency tests protect balances and consistency. Test results are repeatable, traceable and measured.

Examples: simulate a failure after the debit and before the credit to check rollback; run two simultaneous withdrawals of Rs. 8,000 from Rs. 10,000 and confirm that only one succeeds.

26. A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

Answer:

Identify input ranges from requirements and apply equivalence partitioning. Add edge values: minimum, maximum, just below and just above, empty, null, zero, negative, very long strings and

special characters. Add special conditions: leap-year dates, end of month, maximum file size and maximum list length. Review test cases and update them each release to avoid the pesticide paradox.

Developers often make off-by-one errors with $<$, $\leq$, $>$ and $\geq$, so defects cluster at the edges of ranges. Boundary Value Analysis tests exactly these points with few test cases.

Example: an age field accepts 18 to 60. Tests with 17, 18, 19, 59, 60 and 61 reveal an error where the code accepts 17 or rejects 60. Tests with typical values such as 25 or 40 would pass and hide the defect.

27. A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Answer:

Define one test process (plan, design, implement, execute, report, close) based on ISO/IEC/IEEE 29119 and ISTQB. Use common templates for test plans, test cases and defect reports. Set the same entry and exit criteria and minimum coverage for every module. Use one test management and defect tool, and assign clear roles. Audit and improve the process using models such as TMMi.

Every module gets similar coverage, techniques and documentation, which gives even quality. Metrics are comparable across modules and show weak areas. Work is less dependent on individual testers, and new members learn faster. Reviews and audits are easier.

Example: Billing, Inventory and Reports all use the same templates, BVA and equivalence partitioning for inputs, and the exit criterion "no open critical defects", so defect leakage becomes similar in all modules.

28. A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

Answer:

Define validation rules for every field (type, length, range, format, mandatory). Equivalence partitioning: valid and invalid classes for each field. Boundary value analysis for limits. Negative testing and error guessing: empty values, spaces, special characters, very long text, SQL or script strings, wrong data types and wrong file types. Validate on both client and server sides.

The application is exercised with unexpected data and must respond with a clear error message, not a crash. Security-related inputs (injection attempts) are checked, protecting data. Automated validation tests run in regression so that robustness is kept after changes.

Example: an email field is tested with an empty value, "abc", "a@b", a 300-character string and a script tag. Any input that causes a server error or is accepted wrongly is logged as a defect and fixed.

29. A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer:

Exhaustive testing is impossible: high-risk scenarios were not identified or prioritised. Early testing: requirement and design defects were not caught early. Pesticide paradox: the same tests were reused, so new defects remained undetected. Context dependence: production-like data, environment and load were not tested. Other causes: incomplete coverage, weak regression, untested edge cases and poor defect analysis.

Method: analyse leaked defects in the repository by type, module and phase injected, and ask why each was missed.

Risk-based test design with measured coverage of requirements and code. Early reviews and static testing to remove defects before coding. Automated regression suite that includes every leaked defect. Production-like test environment and data, plus performance and compatibility testing. Clear exit criteria and defect removal efficiency tracking before release.

30. A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

Answer:

Early testing (shift-left): testers join requirement and design reviews; static testing before coding. Defect clustering: use past data to focus on defect-prone modules. Risk-based testing: most effort goes to high-impact features. Continuous testing: unit tests and automated checks on every build in CI. Pesticide paradox: review and refresh test cases regularly.

The cost of fixing a defect increases greatly in later phases, so early fixes save cost and effort. Less rework is needed in design, code and test documents. Test execution becomes shorter because major defects are already removed. Release schedules become predictable and quality improves.

Example: a requirement defect found in a review costs minutes to correct. The same defect found after release would need code changes, retesting, redeployment and possibly customer compensation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand